

TCS DRIVE — DevOps Engineer

Interview Preparation Guide

ANSIBLE | AWS | TERRAFORM | KUBERNETES

Candidate: Sanket Ajay Chopade

Role: DevOps Engineer | Company: TCS

SECTION 1 — ANSIBLE INTERVIEW QUESTIONS

Q1.

Tell me about yourself. (Self Introduction)

Structure your answer in 3 parts: Who you are → What you've done → What you bring.

- Background: B.Tech Aeronautical Engineering, RTM Nagpur University (2025).
- Experience: ~9 months internship at Hisan Labs — worked on AWS (EKS, EC2, VPC, RDS, IAM), Terraform, Jenkins, GitHub Actions, Kubernetes, Helm, Docker, Datadog.
- Project: Built ApnaKart — a 10-microservice cloud-native e-commerce platform on AWS EKS using Terraform, Jenkins CI/CD, Kustomize, Trivy, SonarQube.
- Key metrics: 77% reduction in deployment time, 99.9% uptime maintained, zero critical CVEs.

Keep it under 90 seconds. End with: 'I'm looking to grow in infrastructure automation and cloud operations at scale.'

✓ *Tip: Don't list every tool. Speak in achievements, not just technologies.*

Q2.

Have you worked on Linux, Windows, patching, and troubleshooting?

Linux (Strong):

- Package management: yum, apt, rpm.
- Service management: systemctl start/stop/status/enable.
- Log analysis: journalctl, /var/log/, tail -f.
- Shell scripting: Bash automation for deployments and health checks.
- Patching: Used Ansible playbooks to automate OS-level patching across EC2 nodes.

Windows (Conceptual):

- Understand WinRM-based Ansible connectivity for Windows targets.
- Familiar with win_updates, win_service, win_package modules.
- Hands-on has been Linux-primary; Windows knowledge is theoretical.

✓ *Tip: Be honest about Windows. Saying 'conceptual but hands-on Linux' is far better than bluffing.*

Q3.

What is Ansible and what work did you do in Ansible?

Ansible is an open-source, agentless IT automation tool that uses SSH (Linux) or WinRM (Windows) to push configuration to target hosts. No agent installation needed on remote machines.

Core concepts:

- Inventory: defines target hosts (static or dynamic).
- Playbooks: YAML files defining automation tasks.
- Modules: pre-built units of work (yum, copy, template, service, etc.).
- Roles: reusable, structured playbook components.
- Variables: group_vars, host_vars, vault for secrets.

My work:

- Automated application deployments across EC2 instances.
 - Used group_vars and host_vars for environment-specific configs (dev/uat/prod).
 - Wrote reusable roles for package installation and service management.
 - Integrated Ansible with Jenkins pipelines for post-deploy config management.
-

Q4.

Install an RPM file on multiple servers using a template and YAML playbook.

Step 1 — Inventory File (hosts.ini):

```
[webservers] 192.168.1.10 192.168.1.11 192.168.1.12
```

Step 2 — Playbook (install_rpm.yml):

```
--- - name: Install RPM on multiple servers hosts: webservers become: yes vars: rpm_path:
/tmp/mypackage.rpm tasks: - name: Copy RPM to remote ansible.builtin.copy: src:
mypackage.rpm dest: '{{ rpm_path }}' - name: Install the RPM ansible.builtin.yum: name: '{{
rpm_path }}' state: present - name: Deploy config from template (Jinja2)
ansible.builtin.template: src: myconfig.j2 dest: /etc/myapp/config.conf
```

Key distinction — copy module = static file transfer. template module = Jinja2 dynamic rendering with variables.

Q5.

Do you have hands-on experience with Windows services — start/stop/install packages?

Ansible uses win_service module for Windows service management and WinRM for connectivity.

Start / Stop a service:

```
- name: Start Windows Time service win_service: name: W32Time state: started - name: Stop a
service win_service: name: Spooler state: stopped
```

✓ *Tip: For package installation on Windows, use win_chocolatey (Chocolatey package manager) or win_package for .exe/.msi installers.*

Q6.

Check all service status on Windows servers using Ansible — which module?

Use win_service_info module to retrieve status of all or specific services.

```
- name: Get all Windows service statuses win_service_info: register: svc_info - name:
Display results debug: var: svc_info.services
```

✓ *Tip: win_service_info returns a list of service objects with name, state, start_mode, etc. Filter by state == 'running' for active services.*

Q7.

Have you worked on Windows ACLs, Group Policies, or User Policies?

Ansible Windows modules for access control:

- `win_acl` — manages file/folder ACL permissions (grant/deny read, write, execute).
- `win_user` — creates and manages local Windows user accounts.
- `win_group_membership` — adds/removes users from local groups.
- `win_regedit` — used for registry-based Group Policy modifications.
- `win_dsc` — integrates with PowerShell DSC resources for advanced policy enforcement.

Honest note: My hands-on has been Linux-centric. I understand these modules conceptually and can implement from documentation, but deep Windows GP hands-on is not part of my current experience.

✓ *Tip: Always disclose gaps honestly at TCS. They value accuracy over bluffing.*

Q8.

Checking service status, installing packages — any exposure to AWS modules?

Ansible AWS modules fall under the `amazon.aws` and `community.aws` collections (use `boto3` under the hood).

Common modules used:

- `ec2_instance` — provision and manage EC2 instances.
- `s3_bucket` — create/manage S3 buckets.
- `ec2_security_group` — manage inbound/outbound rules.
- `iam_role` — create and attach IAM roles.

My AWS experience (primary via Terraform + CLI):

- EKS cluster management, EC2 auto-scaling, RDS MySQL, CloudFront CDN, Route53 DNS.
 - IAM roles with IRSA/OIDC for pod-level AWS permissions in Kubernetes.
 - ECR for container image registry in uat/prod environments.
-

Q9.

Have you worked with AWS SSM (Systems Manager)?

AWS Systems Manager provides operational control over EC2 instances without opening SSH.

Key features:

- Session Manager: browser/CLI shell access to EC2 without SSH or bastion host. Port 22 not required.
- Parameter Store: secure hierarchical storage for config values and secrets. Integrates with Lambda, ECS, and Kubernetes via External Secrets Operator.
- Run Command: execute shell scripts or commands across a fleet of instances simultaneously.
- Patch Manager: automates OS patching with configurable maintenance windows and compliance reporting.

My usage: Used Parameter Store to store environment-specific DB credentials and API keys, referenced in Terraform using `aws_ssm_parameter` data source — avoiding hardcoded secrets in code.

✓ *Tip: SSM Session Manager is the modern replacement for SSH bastion hosts. Know this positioning.*

SECTION 2 — DEVOPS INTERVIEW QUESTIONS

Q1.

Self Introduction (DevOps Focus)

Same 90-second structure, but emphasize DevOps toolchain and cloud architecture.

- I'm Sanket Chopade, a fresher DevOps Engineer with ~9 months of internship at Hisan Labs.
- Built ApnaKart: a 10-microservice cloud-native e-commerce platform on AWS EKS.
- Tech stack: Terraform (IaC), Jenkins + GitHub Actions (CI/CD), Kubernetes + Helm + Kustomize, Docker, Datadog APM, Trivy, SonarQube.
- Achieved: 77% deployment time reduction, 30% MTTR improvement, 99.9% uptime.
- Seeking to grow in platform engineering and SRE at scale.

Q2.

Do you have experience with Amazon Connect or Lambda?

Amazon Connect: AWS-managed contact center service. No hands-on experience.

AWS Lambda (hands-on):

- Event-driven serverless functions — triggered by S3 uploads, API Gateway, CloudWatch Events (scheduled jobs).
- Written Lambda functions in Python using boto3 for AWS automation.
- Used Lambda for lightweight backend operations: automated EC2 health checks, log processing, and SNS alerting.
- Understand cold start behaviour, memory/timeout configuration, and IAM execution roles.

✓ *Tip: Be direct about Amazon Connect — they won't penalize honesty on a niche service.*

Q3.

EC2 is running but not reachable via SSH — how do you troubleshoot?

Approach systematically — Layer by Layer:

1. Security Group: Check if port 22 (SSH) is open for your source IP. Most common cause.
2. Network ACL: Stateless — check BOTH inbound (port 22) AND outbound (ephemeral ports 1024-65535) rules.
3. Public IP: Verify the instance has a Public IP or Elastic IP assigned.
4. Route Table: Confirm the subnet's route table has a route to an Internet Gateway (0.0.0.0/0 → igw-xxx).
5. Host-based Firewall: iptables or ufw rules inside the OS may be blocking SSH.
6. SSH Service: Check if sshd is actually running inside the instance.
7. Key Pair: Verify you're using the correct .pem key and correct username (ec2-user, ubuntu, etc.).
8. Alternate Access: Use EC2 Instance Connect or SSM Session Manager to get shell access without SSH.
9. Diagnostics: EC2 Console → Get System Log / Get Instance Screenshot for boot errors.

■ *Watch out: Jumping straight to 'check Security Group' without mentioning NACL, route tables, or IGW will lose marks.*

Q4.

What is the difference between a Security Group and a Network ACL?

Security Group vs Network ACL — Key Differences:

Feature	Security Group	Network ACL
---------	----------------	-------------

Operates at	Instance level	Subnet level
State	Stateful	Stateless
Rule types	Allow only	Allow + Deny
Rule evaluation	All rules checked	Numbered order (lowest first)
Return traffic	Auto-allowed	Must explicitly allow
Default	Deny all inbound	Allow all (default NACL)

✓ *Tip: The stateful vs stateless distinction is what interviewers test. Stateful = SG remembers the connection and auto-allows return traffic. Stateless = NACL evaluates every packet independently.*

Q5.

AWS costs suddenly increased — how do you investigate and fix it?

Investigation steps:

1. AWS Cost Explorer: Filter by service, linked account, region, and time range to isolate the spike.
2. Cost Anomaly Detection: Check if any anomaly alerts were triggered automatically.
3. Common culprits: Unexpected EC2 instances, NAT Gateway data transfer spikes, S3 GET/PUT explosion, forgotten RDS instances, unattached EBS volumes.
4. CloudTrail: Audit who provisioned what resources and when — links cost spike to a specific actor.
5. Trusted Advisor: Flags idle EC2 instances, unassociated Elastic IPs, underutilized RDS.

Prevention going forward:

- Set AWS Budgets with SNS alerts for threshold breaches.
- Enforce resource tagging policies (Owner, Environment, Project) for cost attribution.
- Use SCPs to restrict expensive resource types in non-prod accounts.
- Schedule non-prod instances to stop outside working hours (Lambda + EventBridge).

Q6.

Terraform apply failed midway — is re-apply the best option or a bad option?

It depends — and this answer separates seniors from beginners.

What happens during a failed apply:

- Terraform writes partial state — some resources created and recorded, others not.
- Resources created but NOT in state will cause duplicate creation on re-apply.

Correct approach:

1. Run terraform plan first — review what Terraform intends to do.
2. Check state file: terraform state list — see what was actually created.
3. If a resource exists in AWS but not in state: run terraform import to reconcile.
4. If failure was transient (timeout, rate limit): re-apply is usually safe.
5. If failure was a config error: fix the .tf file first, then re-apply.
6. Never blindly re-apply without a plan review — risk of duplicate resources or state corruption.

■ *Watch out: Saying 'just re-apply' without this reasoning will flag you as a beginner. Show the thought process.*

Q7.

Structure Terraform for multiple environments (dev, test, prod)?

Two approaches — know both, advocate for directory-based:

Option A — Workspaces (simple, not recommended for prod):

```
terraform workspace new dev terraform workspace new test terraform workspace new prod
terraform workspace select prod
```

Option B — Directory-based structure (recommended):

```
terraform/ ■■■ modules/ ■ ■■■ vpc/ ■ ■■■ eks/ ■ ■■■ rds/ ■■■ environments/ ■ ■■■ dev/
■ ■ ■■■ main.tf ■ ■ ■■■ variables.tf ■ ■ ■■■ terraform.tfvars ■ ■■■ test/ ■ ■■■ prod/
```

Why directory-based? Each environment gets its own state file, own tfvars, and independently calls shared modules. With workspaces, a wrong 'workspace select' can deploy infra to the wrong environment — dangerous in prod.

Q8.

Terraform drift — how to detect it and what command to use?

Drift = when real infrastructure differs from what Terraform's state file expects (manual console changes, external scripts, etc.).

Detection commands:

- terraform plan — shows any differences between state and real infra. Output shows what would be changed.
- terraform plan -refresh-only — safe way to see drift without applying anything. Only updates state to reflect real infra.
- terraform refresh — forces state update to match reality (use carefully — can mask problems).

Prevention strategies:

- Enforce policy: all infra changes must go through Terraform (IaC-only culture).
- Use IAM SCPs or restrictive IAM policies to block direct console modifications in prod.
- Run scheduled terraform plan in CI/CD pipeline and alert on non-zero diff (drift detection pipeline).
- Tag all resources with ManagedBy=Terraform for visibility.

✓ *Tip: Know 'terraform plan -refresh-only' specifically — it's the modern safe approach. Saying just 'terraform plan' is acceptable but -refresh-only shows deeper knowledge.*

Q9.

Deployment rollout is stuck — what do you check?

Assuming Kubernetes deployment (most common context in DevOps interviews):

Step 1: Check rollout status:

```
kubectl rollout status deployment/
```

Step 2: Check pod states:

```
kubectl get pods -n
```

Step 3: Describe the stuck pod (Events section is key):

```
kubectl describe pod -n
```

Step 4: Check logs from crashed container:

```
kubectl logs --previous -n
```

Common causes and fixes:

- ImagePullBackOff: Wrong image tag, ECR auth issue, or image doesn't exist.
- CrashLoopBackOff: App is crashing on startup — check logs for stack trace.
- Pending: Node resource exhaustion (CPU/memory). Check: `kubectl describe node`.
- Readiness probe failing: App not healthy — rollout waits for Ready state. Fix the probe or the app.
- PVC not bound: Persistent volume not provisioned. Check storage class.

If rollout needs to be reverted:

```
kubectl rollout undo deployment/
```

✓ *Tip: Always check 'kubectl describe pod' Events before anything else — it tells you exactly what failed.*

Q10.

What are your future aspirations?

Answer with direction and authenticity — not flattery toward TCS.

Suggested response:

- Short-term: Go deep on platform engineering — building internal developer platforms that make deployments frictionless for development teams.
- Certifications: AWS Solutions Architect Associate is my immediate target, followed by CKA (Certified Kubernetes Administrator).
- Medium-term: Move into SRE — define SLOs, build error budgets, and own reliability at scale.
- Long-term: Work on large-scale distributed systems in global markets (Germany or USA are aspirational targets).
- I want to grow from executing automation to designing the systems that enable automation at an org-wide level.

✓ *Tip: Don't say 'I want to grow with TCS.' Say where you want to go technically. Ambition is respected.*

KEY GAPS TO ADDRESS BEFORE THE INTERVIEW

1. Ansible + Windows: Memorize `win_service`, `win_service_info`, `win_acl` modules cold. 2. Terraform mid-apply failure: Walk through `plan` → `state list` → `import logic` — don't just say 're-apply'. 3. AWS SSM: Know all 4 features (Session Manager, Parameter Store, Run Command, Patch Manager). 4. Drift detection: Know 'terraform plan -refresh-only' specifically. 5. K8s rollout troubleshooting: Practice the `kubectl describe` → `logs` → `rollout undo` flow.